

Conversation Repairer + Relationship Context Memory

Build guide for Antigravity / AI coding agents. Hacktoberfest 2026 DEV Weekend Challenge: Build for a Friend.

Hard deadline: **Oct 5, 2026, 12:25 PM IST**. Target submission: Mon 08:30 IST. Solo developer, MERN-comfortable, vibe-coding workflow.

0. INSTRUCTIONS FOR THE CODING AGENT (READ FIRST)

- You are implementing the MVP described here. Work **one phase at a time**. After each phase: run it, test it, show the result, and wait for the developer to confirm before the next phase.
- Do **not** add anything outside this document. No auth, no database, no vector DB, no WhatsApp integration, no agents, no fine-tuning, no extra services. If unsure, choose the simpler option.
- Use: React + Vite + Tailwind CSS (client), Node.js + Express (server), plain JavaScript (ES modules). No TypeScript. Keep dependencies minimal.
- The server is **stateless**. It stores nothing and never logs chat content. All user data lives in browser localStorage.
- The raw chat file is parsed in the browser and is never stored. Only extracted memories (short quote evidence) are saved.
- The AI must never claim certainty about anyone's emotions. Use 'possible', 'might', 'could'. This is a communication assistant, not a therapist and not a mind reader.
- Never invent relationship facts. Every memory needs an exact quote that is verified in code against the source chunk.
- Model: Gemma 3 (open-weight). Put ALL instructions inside the user message (some hosted Gemma endpoints do not support system instructions or JSON mode). Parse JSON yourself.
- Never commit .env, API keys, or real chat data. Only synthetic samples go in /samples.
- Keep UI copy in simple English. Keep code readable with short comments.

Product principle

This is **not** 'ChatGPT for relationships'. It is a context-aware conversation repair tool. The differentiator is the pipeline:

```
Exported chat -> Relationship-specific memory (with quotes) -> Relevant context retrieval -> Gemma -> Context-aware repair
```

Assumptions to verify before relying on them: the current Hacktoberfest rules and tags, the exact hosted Gemma model ID and free-tier limits, and the availability of newer Gemma versions. The challenge brief says Gemma 3, so stay with it unless the rules say otherwise.

1. Project in one sentence

Upload an exported WhatsApp chat, let Gemma extract evidence-backed relationship memories, then paste a tense new conversation and get Gemma's analysis plus three repair messages grounded in the relevant history.

2. Feasibility

Feasible, if scope stays disciplined. Core technical work is about 10 to 14 hours with AI tools. The real threats are non-code:

- **Friend availability and consent.** The challenge needs a real friend, real use, and real feedback. Message them immediately. Never fabricate feedback.
- **Third-party privacy.** A chat export includes the person your friend talks to. Public material (repo, screenshots, demo) must use synthetic data only.
- **Gemma quality in Hinglish.** Small models are inconsistent. Test on day one.

- **Wrapper perception.** The memory dashboard with evidence quotes is the answer. Make it visibly strong.

3. MVP definition

Tier	Items
MUST	WhatsApp .txt parser (Android + iOS), chunked Gemma memory extraction with evidence, memory dashboard with edit/delete, paste conversation, keyword retrieval, repair analysis with 3 styles, JSON validation/repair, Delete Everything, privacy notice, deployed link, README, synthetic sample chats, DEV article
SHOULD	Confidence badges, 'show evidence' toggle, copy button, English/Hinglish output toggle, extraction progress indicator
NICE	Export/import memories as JSON, regenerate one style, dark mode
CUT	Auth, MongoDB, embeddings, multi-chat/multi-contact, WhatsApp integration, mobile app, streaming, fine-tuning, agents, analytics

4. Recommended architecture

```

BROWSER (React + Vite + Tailwind)
+-----+
| Upload -> parse .txt (client-side) |
| Chunk messages |
| Memory store (localStorage) <- Dashboard |
| Keyword retrieval (client-side) |
+-----+
| only text chunks / pasted convo + top memories |
v
EXPRESS (stateless, no database, no content logs)
+-----+
| POST /api/extract -> prompt + chunk |
| POST /api/repair -> prompt + convo + mems |
| safeJson parse / validate / retry |
| llm.js: provider switch (hosted | ollama) |
+-----+
v
Gemma 3 (hosted API, or local Ollama)

```

Honest privacy statement: the raw chat file is parsed in your browser and never uploaded as a file. Text chunks are sent to the model for analysis. The server stores nothing. Do NOT claim 'data never leaves your device' for the hosted version.

5. Technology decisions

A. Local vs hosted Gemma: hosted for demo, Ollama as documented option

	Local (Ollama)	Hosted API
Setup	Fast if hardware allows	Fast (API key)
Reliability	Depends on machine	Rate limits / quota
Privacy	Best	Chat text goes to provider
Deploy a demo	Hard (needs GPU)	Easy
Hackathon risk	High for deployed demo	Medium

Build one `llm.js` with `LLM_PROVIDER=hosted|ollama`. Start with Google AI Studio Gemma 3 or OpenRouter, whichever gives a key fastest. If a provider fails for 45 minutes, switch providers.

Model size: hosted gemma-3-27b-it (better Hinglish and JSON discipline); fall back to 12B if rate-limited. Local: gemma3:4b on any laptop, gemma3:12b with about 16GB+ memory.

B. Storage: browser localStorage

No MongoDB. Memories are small, there are no accounts, deletion is honest, deployment stays trivial.

C. Retrieval: keyword scoring

Gemma generates keywords per memory at extraction time (include Hinglish/English variants). At query time: tokenize the pasted text, score by overlap, boost categories recurring_issue/promise/preference, add a small recency bonus, return top 6 to 8. Embeddings are V2.

D. Chunking

About 120 messages per chunk (cap about 5,000 characters) with 10-message overlap. Cap the MVP at the most recent ~1,500 messages (about 12 chunks) and tell the user. Process 2 chunks concurrently. Small chats (under ~150 messages) can be one chunk.

6. User flow

```
Upload .txt -> parse (browser) -> show "N messages, date range, senders"
-> user selects which sender is "me" -> chunk -> POST /api/extract per chunk
-> merge + dedupe -> save to localStorage -> Memory Dashboard (edit / delete)
-> Repair tab: paste convo -> local retrieval (top 6-8) -> POST /api/repair
-> validated JSON -> Results: issue, context + evidence, uncertainty, avoid,
    3 messages -> user edits / copies
```

Screens

Screen	Contents
1 Upload	Title 'Conversation Repairer', tagline 'Turn your conversation history into useful context.', file picker, privacy notice, 'How to export WhatsApp chat' help
2 Processing	Progress: Parsed messages / Extracting context (chunk x of N) / Organizing memories
3 Memory Dashboard	Sections: Important, Preferences, Communication Patterns, Promises, Recurring Issues, Meaningful References. Each memory card: text, evidence quote, confidence badge, Edit, Delete. Always-visible 'Delete everything' button.
4 Repair	Textarea 'Paste a difficult conversation', 'Analyze Conversation' button, note about truncation if very long
5 Results	Possible communication issue; Evidence in text; Relevant context (memory + quote + why relevant, or 'No relevant history found'); Uncertainty; What to avoid; three cards Soft / Casual / Direct, each editable with a Copy button

Design: warm palette, card-based sections, an evidence drawer ('Why this context?'). Must not look like a generic ChatGPT clone.

7. Implementation phases

Rule for every phase: BUILD, RUN, TEST, VERIFY, then continue. Do not start the next phase until the acceptance criteria pass.

Phase 0: Setup and skeleton (45 min)

- Goal: both servers run and one Gemma call works.
- Build: Vite + React + Tailwind client; Express server with CORS and JSON body limit 1mb; .env loading; llm.js; GET /api/health.
- Files: server/index.js, server/llm.js, .env.example, client scaffolding.
- Acceptance: GET /api/health returns ok; a test prompt returns text from Gemma. If Gemma does not respond within 45 minutes, switch provider.

```
# .env.example
LLM_PROVIDER=hosted          # hosted | ollama
GEMMA_API_KEY=your_key_here
GEMMA_MODEL=gemma-3-27b-it   # verify exact ID with your provider
OLLAMA_URL=http://localhost:11434
OLLAMA_MODEL=gemma3:4b
PORT=8787
```

llm.js contract: export async function generate(prompt, {temperature=0.2, timeoutMs=45000}) -> string. Handles provider switch, timeout, and one retry on 5xx. On HTTP 429 back off 2s then 5s.

Phase 1: WhatsApp parser (1.5 h)

FEATURE	WhatsApp Chat Parser
Goal	Convert exported WhatsApp .txt into structured messages.
Files	client/src/lib/whatsappParser.js
Input	Raw text of exported .txt file
Processing	Detect format per line with regex. Android: 12/03/2026, 21:45 - Name: text. iOS: [12/03/26, 9:45:12 PM] Name: text. Strip invisible chars (\u200e, \u200f). Lines not matching a header are continuation of the previous message (multiline). Skip system lines (encryption notice, 'X added Y', 'changed the subject', etc.; lines with a timestamp but no 'Name:' part). Keep media placeholders as text like [media] (<Media omitted>, 'image omitted'). Support 12h and 24h times, dd/mm and mm/dd.
Output	{ messages:[{id,date,time,sender,text}], senders:[], stats:{count,firstDate,lastDate} } or a thrown Error with a readable message.
Dependencies	None.
Acceptance criteria	Parses normal messages; handles multiline; ignores system lines; handles media placeholders; handles both formats; empty or non-WhatsApp file returns 'Couldn't find WhatsApp messages in this file'; verified on 3 sample files.

Phase 2: Chunker + extract endpoint (2 h)

FEATURE	Chunker and Memory Extraction
Goal	Gemma returns valid, verified memory JSON for one chunk.
Files	client/src/lib/chunker.js, client/src/lib/api.js, server/routes/extract.js, server/prompts/extract.js, server/lib/safeJson.js
Input	Formatted chunk text (lines like 'Name: text'), names of ME and FRIEND

Processing	Build prompt (Section 8). Call generate() at temperature 0.2. safeJson: strip code fences, take text between first '{' and last '}', JSON.parse; on failure retry once with 'Return ONLY valid JSON'; if still failing return {memories:[]} plus a warning. Validate each memory: category in allowed list, memory is non-empty string, evidence is a string, confidence in high medium low. Drop any memory whose evidence is not a substring of the chunk (normalize whitespace and case). Client side: process 2 chunks concurrently, collect warnings.
Output	{ memories:[{category,memory,evidence,speaker,keywords,confidence}], warnings:[] }
Dependencies	Phase 0, Phase 1
Acceptance criteria	On the Hinglish sample chat at least 70% of memories carry a quote verified in the chunk; invalid JSON never crashes the server; a failed chunk is skipped with a warning, others continue.

Phase 3: Consolidation, storage, dashboard (2.5 h)

FEATURE	Memory Consolidation and Dashboard
Goal	Merge duplicate memories, persist them, and let the user view/edit/delete.
Files	client/src/lib/dedupe.js, client/src/lib/memoryStore.js, components/UploadScreen.jsx, ProcessingScreen.jsx, MemoryDashboard.jsx, MemoryCard.jsx, App.jsx
Input	Raw memories from all chunks
Processing	Normalize text (lowercase, strip punctuation); same category and token Jaccard similarity above 0.6 means merge: keep highest confidence, keep up to 2 evidence quotes, union keywords. Assign ids m_001... Save to localStorage under 'cr:memories'. Group by category. Inline edit sets userEdited=true. Delete removes one. Delete Everything clears localStorage and returns to Upload. Low-confidence memories render muted.
Output	Dashboard with six category sections.
Dependencies	Phase 2
Acceptance criteria	Six sections render; edit text inline; delete one; Delete Everything clears storage; page refresh keeps memories; empty state shown when no memories.

Phase 4: Retrieval and repair endpoint (2.5 h)

FEATURE	Retrieval and Context-Aware Repair
Goal	Retrieve relevant memories and have Gemma produce a structured repair analysis.
Files	client/src/lib/retrieve.js, server/routes/repair.js, server/prompts/repair.js, components/RepairScreen.jsx, ResultsScreen.jsx
Input	Pasted conversation text; stored memories; ME and FRIEND names
Processing	retrieve.js: tokenize (lowercase, strip punctuation, drop stopwords incl. Hinglish: hai, ka, ki, ko, to, toh, ne, se, bhi, nahi, kya, me, mein, ho, tha, thi). Score = keyword overlap (memory.keywords and memory text) + category boost + small recency bonus. Exclude low-confidence by default. Return top 8; if best score is 0 send zero memories. Truncate pasted text to the last ~6,000 chars and show a notice. Server builds repair prompt (Section 8) with memories as {id,text,evidence}; temperature 0.7 for the first try, 0.2 on retry.
Output	Validated repair JSON (see Section 9).
Dependencies	Phase 3
Acceptance criteria	For the cancellation test case the cancellation memory is retrieved and shown with its quote; for the unrelated test case Results shows 'No relevant history found' and no invented context.

Phase 5: Output validation and errors (1.5 h)

FEATURE	Validation and Error Handling
Goal	Never show broken or unsafe output.
Files	server/lib/validateRepair.js, client toast/error components

Input	Raw model output
Processing	Require fields: possible_issue (string), evidence_in_text (array), relevant_context (array), uncertainty (string), avoid (array), suggestions (exactly 3: soft, casual, direct, strings only). Trim message length to about 60 words. Drop relevant_context entries whose memory_id was not sent. Fill missing fields with safe defaults. Scan outputs for certainty phrases ('is angry', 'she feels', 'he hates') and soften or flag. Map errors per the table in Section 8.
Output	Always a safe, renderable object or a friendly error with a Retry button.
Dependencies	Phase 4
Acceptance criteria	Malformed JSON, timeout, 429 and empty input each produce a friendly message and the app stays usable.

FEATURE FREEZE happens after Phase 5 (Sun 17:00 IST). No new features after that.

Phase 6: UI polish and privacy (1.5 h, after freeze)

- Warm palette, card layout, evidence drawer, loading states, mobile-friendly widths.
- Privacy notice on upload screen; 'Delete everything' always visible; text states what is sent to the model and what is stored.
- Acceptance: a first-time user can complete the flow without instructions.

Phase 7: Deploy (1.5 h)

- Client and server together on Render/Railway/Fly, or client on Vercel/Netlify and server on Render. Set env vars on the host. Enable CORS only for the client origin.
- Test the full flow on the deployed URL in a fresh browser with the synthetic chat. If deploy breaks late, record the demo from localhost.

8. Gemma design

Gemma does three jobs: (1) extract memories from chunks, (2) analyze the pasted conversation using retrieved memories, (3) write repair messages in the user's own style. Code does parsing, retrieval, dedupe, and validation.

Extraction prompt (server/prompts/extract.js)

You extract relationship memories from a WhatsApp chat excerpt between "{ME}" and "{FRIEND}". The chat may be English, Hindi, Romanized Hindi, or Hinglish, with slang, abbreviations and emojis.

Extract ONLY things useful for future communication. Categories: important_event, preference, communication_pattern, promise, recurring_issue, meaningful_reference.

Rules:

- Every memory MUST include an exact quote copied from the chat as evidence. If you can't quote it, don't include it.
- Do not infer emotions or motives. State only what is said or clearly done.
- Write each memory in simple English; keep key Hinglish phrases in the quote as-is.
- Include 3-6 "keywords" (lowercase; include Hinglish and English variants).
- confidence: high = stated directly; medium = repeated pattern; low = weak hint.
- If nothing useful exists, return {"memories": []}.
- Return ONLY valid JSON, no markdown, no commentary.

Schema: {"memories":[{"category":"...", "memory":"...", "evidence":"exact quote", "speaker":{"name","keywords":["..."],"confidence":"high|medium|low"}]}

CHAT:
{chunk}

Repair prompt (server/prompts/repair.js)

You are a communication assistant, not a therapist and not a mind reader. A user pasted a conversation they are worried about. Help them repair or continue it.

Language: Understand English, Hindi, Romanized Hindi and Hinglish. Write suggestions in the SAME style as the user's messages. Do NOT use formal/shuddh Hindi. Keep it natural and casual, like real texting. If the user writes Hinglish, write Hinglish.

Rules:

- Never claim certainty about what someone feels or thinks. Use "might", "could", "possible".
- Separate what is visible in the text (evidence) from your interpretation.
- Use ONLY the provided memories as history. Cite them by id. If none are relevant, say so; do not invent history.
- Do not blame either person. No clinical or diagnostic language.
- If the conversation shows safety concerns (threats, self-harm), set "safety_note" and keep suggestions minimal and kind.
- 3 suggestions: "soft" (warm, gentle), "casual" (short, natural), "direct" (honest, clear, still kind).
- Messages must be sendable as-is, under 60 words each, no placeholders like [name].

Return ONLY valid JSON:

```
{"possible_issue":"","evidence_in_text":["quote"],"relevant_context":[{"memory_id":"","why_relevant":""}],
"uncertainty":"","avoid":[""],"safety_note":"","suggestions":[{"style":"soft","message":""}, {"style":"casual","message":""}, {"style":"direct","message":""}]}
```

MEMORIES: {list of {id, text, evidence}}

USER IS: {ME}. OTHER PERSON: {FRIEND}.

CONVERSATION: {paste}

Hinglish handling

- Prompt explicitly lists English / Hindi / Romanized Hindi / Hinglish, and says to mirror the user's style and avoid formal Hindi.
- Memories are written in simple English; quotes stay in the original language.

- Keywords include Hinglish variants so retrieval works on mixed text (e.g. cancel, plan, reply, call, kal, late).
- Example: input 'Bhai mujhe lag raha hai woh thodi upset hai but idk' should produce casual Hinglish suggestions, not formal Hindi or stiff English.
- If quality is poor: add 2 few-shot Hinglish examples to the repair prompt, or use the 27B model.

Hallucination prevention (in code)

- Evidence must be an exact (normalized) substring of the chunk, otherwise drop the memory.
- Repair prompt receives only memory ids + text + evidence; output must reference ids; unknown ids are dropped.
- Low-confidence memories are muted in the UI and not sent to repair by default.
- 'uncertainty' field is mandatory; UI always displays it.
- Use temperature about 0.2 for extraction, about 0.7 for repair.

Error handling table

Case	Behavior
Invalid / unsupported file	Parser error with link to export help
Empty chat	'No messages found'
Huge chat	Use last ~1,500 messages; tell user
AI timeout / unavailable	45s timeout, 1 retry, then retry button; keep partial extraction progress
Malformed JSON	safeJson + one retry; else skip chunk with warning
Rate limit (429)	Backoff 2s then 5s; concurrency 2
Hallucination	Evidence substring check; drop unquoted memories
Duplicate memories	Jaccard dedupe
Irrelevant memories	Score threshold; show 'No relevant history found'
Very long pasted conversation	Truncate to last ~6,000 chars with notice
Sensitive content	Privacy notice before upload; safety_note path in repair output

9. Data model

localStorage key 'cr:memories'

```
{
  "meta": { "me": "Aman", "friend": "Riya", "messageCount": 1480, "importedAt": "2026-10-04T10:00:00Z" },
  "memories": [
    {
      "id": "m_001",
      "category": "recurring_issue",
      "memory": "Riya dislikes last-minute plan cancellations",
      "evidence": ["yaar last moment pe cancel mat kiya kar"],
      "speaker": "Riya",
      "keywords": ["cancel", "plan", "last minute", "cancel mat"],
      "confidence": "high",
      "userEdited": false
    }
  ]
}
```

Raw chat is **not** stored. Say this in the UI.

Repair response (validated)

```
{
  "possible_issue": "...",
  "evidence_in_text": ["..."],
  "relevant_context": [ { "memory_id": "m_001", "why_relevant": "..." } ],
  "uncertainty": "...",
  "avoid": ["..."],
  "safety_note": "",
  "suggestions": [
    { "style": "soft", "message": "..." },
    { "style": "casual", "message": "..." },
    { "style": "direct", "message": "..." }
  ]
}
```

API contracts: POST /api/extract {chunk, me, friend} -> {memories, warnings}. POST /api/repair {conversation, memories:[{id,text,evidence}], me, friend} -> repair JSON. Body limit 1mb.

10. Folder structure

```
conversation-repairer/
|-- client/
|   |-- src/
|   |   |-- components/ (UploadScreen, ProcessingScreen, MemoryDashboard, MemoryCard,
|   |   |               RepairScreen, ResultsScreen)
|   |   |-- lib/ (whatsappParser, chunker, dedupe, memoryStore, retrieve, api)
|   |   |-- App.jsx
|   |-- server/
|   |   |-- index.js
|   |   |-- llm.js
|   |   |-- routes/ (extract.js, repair.js)
|   |   |-- prompts/ (extract.js, repair.js)
|   |   |-- lib/ (safeJson.js, validateRepair.js)
|   |-- samples/ (synthetic chats + expected results)
|   |-- README.md
|   |-- LICENSE      (MIT or Apache-2.0)
|   |-- .gitignore   (.env, node_modules, real-chats/)
|   |-- .env.example
```

11. Test plan

Create all samples as **synthetic** data in /samples (about 15 minutes with an AI tool).

#	Case	Expected behavior
1	Normal English chat (~80 msgs)	Parses; 3 to 8 memories, all with real quotes
2	Hinglish chat	Memories in simple English, quotes in Hinglish; repair output in Hinglish
3	Long chat (1,500+ msgs)	Chunking runs, progress shown, dedupe merges repeats, no crash
4	Obvious preference ('mujhe calls pasand hain, texting nahi')	One preference memory, high confidence
5	Promise ('exam ke baad call karunga')	promise memory
6	Conflict (plans cancelled twice)	recurring_issue memory; retrieved when a new cancellation is pasted
7	Short cold reply ('k.') with no stated emotion	Output says 'possible', names uncertainty, NEVER 'she is angry'
8	Pasted conversation unrelated to history	'No relevant history found'; no invented context
9	Android and iOS formats + a multiline message	Both parse correctly
10	Garbage .txt	Friendly error

Manual end-to-end test

- Upload sample, compare dashboard with the file, delete one memory, paste case 6, confirm the cancellation memory appears with its quote, copy a message, click Delete Everything, refresh, confirm empty.
- During prompt tuning grep outputs for banned certainty words ('is angry', 'she feels', 'he hates you').

12. Risks that could kill this project before the deadline

Risk	Prob.	Impact	Prevention	Fallback
Gemma setup / API access stalls	Med	Critical	Phase 0 in first hour; provider switch	Switch provider; Ollama 4b
Rate limits during extraction	High	Med	Concurrency 2, backoff, 12-chunk cap	Smaller model; reduce cap
Malformed JSON	High	Med	safeJson + retry + validation	Skip chunk with warning
Hinglish quality poor	Med	High	Test on day 1; explicit style rules	Few-shot examples; 27B model
WhatsApp format edge cases	High	Med	Test Android + iOS + multiline early	Support only tested formats; clear error
Memory extraction generic	Med	High	Require quotes; narrow categories	Tighten prompt; fewer categories
Retrieval misses	Med	Med	Gemma keywords + category boost	Send top-N by confidence
Feels like ChatGPT wrapper	Med	High	Demo dashboard and evidence quotes	Lead article with the pipeline
AI invents relationship facts	Med	Critical	Evidence substring check; ids only	Show only verified memories
Suggestions generic	Med	High	Require use of context; style rules	Add few-shot examples
Friend unavailable / no consent	Med-High	Critical	Message them immediately	Be honest in article; never fake
Private data leaks into repo/demo	Low	Critical	Synthetic data only; .gitignore	Redact; blur screenshots
Open-source AI not clearly shown	Low	Med	State Gemma + open weights + Ollama in README and demo	n/a
Missing tags / article / demo	Med	High	Checklist, finish by Mon 08:00	Short article is fine
Deploy breaks late	Med	High	Deploy a skeleton in Phase 0/1	Record demo from localhost

13. Deadline roadmap (IST)

Target submission: **Mon 08:30**. Hard deadline: **Mon Oct 5, 12:25 PM IST**. Adjust if you start at a different time.

When	Work
Sat now (30 min)	Message friend (consent, export, Sunday evening slot). Create repo. Get Gemma API key. Install Ollama + pull gemma3:4b in background.
Sat evening	Phase 0, Phase 1; start Phase 2; write synthetic samples; deploy a skeleton
Sat night	Phase 2 working on the sample chat; stop at a stable point; sleep
Sun 09:00-13:00	Finish Phase 2; Phase 3 (dedupe, dashboard, edit/delete)
Sun 13:00-16:30	Phase 4, Phase 5 (retrieval, repair, validation)
Sun 17:00	HARD FEATURE FREEZE. Only bug fixes, polish, testing, deploy, docs from here.
Sun 17:00-19:00	Fix bugs, tune prompts on test cases 1-10, UI polish, verify delete/privacy
Sun 19:00-20:30	Deploy final; test on deployed URL
Sun 20:30-22:00	Friend session; record honest reaction with permission
Sun 22:00-23:00	Screenshots (synthetic data), README draft, architecture diagram
Sleep	Required. Tired mistakes cost more than an extra hour of coding.
Mon 06:30-07:30	Record demo (synthetic data); finish README
Mon 07:30-08:30	DEV article, tags, submit
Mon 08:30-12:25	BUFFER ONLY: broken links, edits, resubmission

14. Demo script (60 to 90 seconds)

Time	Action / narration
0:00	'My friend [first name, with permission] often feels conversations go wrong over small misunderstandings, like a cancelled plan or a short reply.'
0:10	Upload an exported chat (synthetic in public demo). 'It's parsed in the browser. Gemma reads it in chunks.'
0:25	Show memory dashboard. Open one memory: 'Every memory has an exact quote. If the model can't quote it, it's dropped.'
0:40	Paste a new tense Hinglish exchange.
0:50	Results: 'Possible issue, not a diagnosis.' Show relevant context with evidence, the uncertainty line, and what to avoid.
1:05	Show soft / casual / direct messages in natural Hinglish. Edit one, copy it.
1:15	'Gemma 3 is open-weight, so this can run fully local with Ollama. That matters for conversations this private.' Click Delete Everything.

15. DEV article structure

Required tags (confirm against the official post): **#devchallenge #weekendchallenge #hf26challenge**. Use the challenge's submission template if one exists.

Section	Collect while building
Friend / problem	Their own words (with permission), anonymized
Why I built it	One honest paragraph
What it does	3 screenshots (synthetic data)
Architecture	Diagram from Section 4
How Gemma is used	The three jobs plus one prompt snippet
Why open models matter	Local-run option, no lock-in, privacy for intimate data
Privacy	What is sent where, what is stored, the delete button; honest about the hosted API
Demo	Video / link
GitHub	Link
What I learned	Real failures: JSON breakage, Hinglish issues, hallucinated quotes caught by validation
Future	Embeddings, local-first mode, multiple chats
Friend's reaction	REAL quotes only, with permission. If they did not get to try it, say so.

Friend session plan

- Get consent for using their chat and for quoting feedback.
- Let them use it on their own device, ideally with their real chat. Do not record the session.
- Ask: 'Did the memories feel accurate?' 'Would you send any of these messages?' 'What felt wrong or creepy?'
- Write notes immediately. Ask what you may publish and how to anonymize it.
- Negative feedback is fine to include and makes the article more credible.

16. Final submission checklist

- [] Parser works for Android and iOS exports
- [] Extract -> dashboard -> edit/delete works
- [] Retrieval returns relevant memories on test cases
- [] Repair returns valid structured output with 3 messages

- [] Hinglish tested; no certainty claims about emotions
- [] Errors handled (file, timeout, JSON, 429)
- [] Delete Everything works
- [] Privacy notice accurate (hosted vs local)
- [] Deployed URL works in a fresh browser
- [] Public repo, MIT/Apache-2.0 license, no real chats or API keys committed
- [] .env.example, setup steps, Ollama instructions, model name
- [] Synthetic sample chats in /samples
- [] Architecture diagram + screenshots in README
- [] Demo video recorded (synthetic data)
- [] Friend tested; permission for feedback recorded
- [] DEV article published with the 3 tags and links to repo and demo
- [] Submission verified before Mon 08:30

README must include

- What it is, why it exists, who it was built for (no private details)
- Setup steps, environment variables, hosted vs Ollama instructions, model name
- Architecture diagram, screenshots, demo link
- Privacy warning and honest data-flow description
- Sample data instructions, license

17. If I fall behind: emergency cut-down

Time left (to Mon 12:25)	Cut
~30 h	Language toggle, confidence badges, dark mode, export/import
~20 h	Dedupe via code only; cap chat at 800 messages; single extraction pass
~14 h	Drop iOS format if it fights you; drop inline edit (keep delete only)
~10 h	Replace scoring with 'top 8 by confidence' plus keyword filter
~7 h	No new code except bugs. Stop prompt tuning. Deploy what works.
~5 h	Record demo, README, article. Use localhost recording if deploy is broken.
~3 h	Fix / deploy / document / submit only. A working smaller project beats a broken larger one.

18. First 5 things to do right now

1. **Message your friend:** explain the project, ask for consent and a chat export, book ~30 min on Sunday evening.
2. **Read the official challenge rules and submission template;** verify Gemma version, tags, and required links.
3. **Get a Gemma API key** (AI Studio or OpenRouter) and run one test call; install Ollama and pull gemma3:4b as fallback.
4. **Create the public repo** (MIT license, .gitignore for .env and real-chats/) and scaffold client + server.
5. **Write two synthetic chats** (one English, one Hinglish, each with a preference, a promise, and a cancellation conflict) so you can test the parser immediately.